CUGG1004 | DATA PREPROCESSING AND EDA

# Time & Text Data

A practical 2-hour lecture on preprocessing, common problems, and reliable workflows

**Module 5 • Advanced Data Handling**

Time series preprocessing + text cleaning

# Where this fits in Module 5



| | |
|---|---|
| **Syllabus focus** | Time-series preprocessing; text cleaning: tokenization, stop-word removal, stemming, lemmatization. |
| **Practical outcomes** | Preprocess time series using resampling and smoothing; implement text preprocessing techniques. |
| **Course outcome** | CO5 asks students to design workflows for time-series, text, and imbalanced datasets effectively. |
| **Scope today** | We cover only time and text, as requested. Imbalanced data, augmentation, Pandas/Dask automation are outside the main lecture. |

TEACHING NOTE: The uploaded syllabus names these exact Module 5 topics and Experiment 5.1–5.2.

# Why preprocessing matters



| | |
|---|---|
| **Raw data** | Collected for operations, not necessarily ready for analysis or machine learning. |
| **Quality** | Errors, missingness, duplicates, inconsistent formats, and noise distort patterns. |
| **Representation** | Models need consistent numeric or token representations; raw timestamps and prose are not usually enough. |
| **Risk** | A wrong preprocessing choice can erase signal, introduce bias, or leak future information. |
| **Principle** | Understand the task and data-generating process before transforming anything. |

# Two data families, two kinds of structure



## TIME SERIES

- Rows are ordered by time; spacing between observations matters.
- Examples: hourly temperature, daily sales, sensor readings, stock prices.
- Questions: what changes, when, how quickly, and what comes next?
- Key risks: gaps, irregular sampling, time zones, drift, future leakage.

## TEXT DATA

- Observations are words, tokens, sentences, documents, or messages.
- Examples: reviews, support tickets, news, survey answers, clinical notes.
- Questions: what is being discussed, expressed, requested, or classified?
- Key risks: noisy language, ambiguity, negation, domain terms, privacy.

# A preprocessing workflow that works for both



| | |
|---|---|
| **1. Define goal** | Forecasting? Anomaly detection? Sentiment? Topic classification? The task determines safe transformations. |
| **2. Inspect** | Schema, examples, data types, ranges, duplicates, missingness, and collection process. |
| **3. Validate** | Check assumptions: chronological order, sampling frequency, language, encoding, label quality. |
| **4. Transform** | Apply task-appropriate cleaning and feature creation; preserve raw data. |
| **5. Evaluate** | Compare before/after summaries and inspect representative examples. |
| **6. Reproduce** | Fit transformations only on training data where needed; document and version the pipeline. |

# Time series: what makes it different?



| | |
|---|---|
| **Time index** | Each observation is tied to a point or interval in time. |
| **Order matters** | Shuffling rows can destroy temporal relationships and invalidate evaluation. |
| **Frequency** | Seconds, minutes, hours, days, months, or irregular event times. |
| **Components** | Trend (long-term direction), seasonality (repeating pattern), cycles, noise, and events. |
| **Multiple series** | Different devices, stores, people, or locations may each have their own timeline. |

TEACHING NOTE: Ask students: why is a random train/test split risky for forecasting?

# Common time-series formats



| | |
|---|---|
| **Wide format** | One timestamp column plus multiple measure columns: time, temperature, humidity. |
| **Long format** | Rows include timestamp, entity ID, variable, and value; useful for many sensors/stores. |
| **Event logs** | A timestamped event stream; events may be irregular and have no value at every interval. |
| **Index vs column** | A timestamp can be a normal column or a DatetimeIndex; choose deliberately. |
| **Granularity** | A daily total is not equivalent to an hourly reading; never assume aggregation is harmless. |

# Read, parse, and sort timestamps



```
import pandas as pd

df = pd.read_csv("sensor_readings.csv")
df["timestamp"] = pd.to_datetime(
    df["timestamp"], errors="coerce", utc=True
)
df = df.dropna(subset=["timestamp"])
df = df.sort_values("timestamp")
df = df.set_index("timestamp")

print(df.index.min(), df.index.max())
print(df.index.is_monotonic_increasing)
print(df.index.duplicated().sum())
```

## WHAT TO NOTICE

- Parse strings into datetime values before time operations.
- errors='coerce' makes invalid timestamps visible as NaT; inspect them before dropping.
- UTC avoids many cross-time-zone ambiguities; convert to local time only when appropriate.
- Sorting is essential before rolling windows, interpolation, and time-based splits.

**Do not silently discard many invalid timestamps: investigate the source.**

# Time-series data problems: diagnose before fixing



| | |
|---|---|
| **Missing timestamps** | Expected intervals are absent: a sensor may have stopped transmitting. |
| **Missing values** | A timestamp exists, but the measurement is null or invalid. |
| **Duplicates** | Repeated timestamp/entity pairs may be retries, repeated measurements, or genuine events. |
| **Irregular intervals** | Events arrive at uneven times; fixed-window methods may need resampling. |
| **Outliers / spikes** | Could be sensor errors—or real events that matter. |
| **Clock problems** | Time zones, daylight-saving changes, inconsistent units, and clock drift can shift observations. |

# Missing data: choose the repair based on meaning



## OPTIONS

- Drop rows only when loss is small and defensible.
- Forward fill: carry last known value; useful for some slowly changing states.
- Backward fill: use next value; often unsafe for real-time prediction.
- Interpolation: estimate between nearby observations; better for smooth signals.
- Resample and aggregate: create a regular grid, then decide how to fill gaps.

## CAUTIONS

- Never fill across long outages without marking the gap.
- Do not treat a missing timestamp as zero unless zero is meaningful.
- Forward fill can create artificial flat periods.
- Interpolation may invent values during abrupt changes.
- Keep a missingness indicator when the fact that data was missing is informative.

# Resampling: change the time frequency



```
            # Assume timestamp is a DatetimeIndex
hourly = df["temperature"].resample("h").mean()
daily_mean = df["temperature"].resample("D").mean()
daily_max = df["temperature"].resample("D").max()

# Inspect how much data contributes to each hour
hourly_count = df["temperature"].resample("h").count()
```

## WHAT TO NOTICE

- Resampling groups observations into time bins and applies an aggregation.
- Downsampling: minute-level data → hourly/daily summaries.
- Upsampling: hourly data → minute grid; creates gaps, not new measurements.
- Choose mean/sum/min/max/last based on the measurement and question.
- Check bin boundaries, time zone, and counts per bin.

**For transaction counts use sum/count; for instantaneous sensor readings, mean or median may fit.**

# Resampling: practical examples and traps



| | |
|---|---|
| **Retail** | Hourly transactions → daily revenue: sum; daily average temperature: mean. |
| **Monitoring** | High-frequency sensor → 5-minute median to reduce isolated spikes. |
| **Finance** | Daily prices → weekly last observed price may be more meaningful than weekly mean. |
| **Uneven sampling** | A simple mean can over-represent periods with many observations; consider time-weighted methods. |
| **Aggregation loss** | Daily averages can hide brief peaks; preserve min/max/count if peaks matter. |

# Smoothing: reduce short-term noise

```
          # Rolling mean: window of 3 observations
df["rolling_mean"] = df["value"].rolling(
    window=3, min_periods=1
).mean()

# Time-based window: previous 30 minutes
df["rolling_30min"] = df["value"].rolling(
    "30min", min_periods=1
).mean()

# Exponentially weighted mean
df["ewm"] = df["value"].ewm(
    span=5, adjust=False
).mean()
```

## WHAT TO NOTICE

- Rolling mean smooths local variation; window size controls the trade-off.
- A time-based window is useful when sampling intervals vary.
- EWMA gives more weight to recent values; span controls responsiveness.
- Trailing windows are generally safer for live forecasting than centered windows.
- Smoothing can hide real spikes, turning points, or anomalies.

**Never smooth automatically: compare the original signal and smoothed version.**

# Outliers, gaps, and anomalies



| | |
|---|---|
| **Start with plots** | Line plots reveal spikes, flatlines, step changes, and outages. |
| **Use context** | Physical limits, operating ranges, and domain rules are more useful than blind thresholds. |
| **Robust summaries** | Median and IQR can help flag extreme values; they do not prove an error. |
| **Action options** | Correct verified errors, cap only with justification, mark anomalies, or retain genuine events. |
| **Document** | Record the rule, count affected, and whether values were changed, removed, or flagged. |

# Time features: useful, but not magic



| | |
|---|---|
| **Calendar** | Hour, day of week, month, weekend, holiday, season. |
| **Elapsed time** | Time since previous observation, since an event, or since process start. |
| **Lags** | Past values such as value(t−1), value(t−7); useful when history predicts the future. |
| **Rolling statistics** | Past-window mean, standard deviation, min/max, counts. |
| **Cyclical encoding** | For hour-of-day or day-of-week, sine/cosine can represent wrap-around. |
| **Caution** | Feature values must be available at prediction time; never use future observations. |

# Prevent time leakage in forecasting



```
                    # Split chronologically
train = df.loc[: "2025-06-30"]
test  = df.loc["2025-07-01":]

# A one-step lag uses a past value
df["lag_1"] = df["value"].shift(1)

# Trailing rolling statistic (past/current only)
df["mean_7"] = df["value"].rolling(7).mean()

# For target prediction, features must be aligned
# to information available at the forecast origin.
```

## WHAT TO NOTICE

- Do not randomly shuffle time series for ordinary forecasting evaluation.
- Fit scalers/imputers on training data only; apply learned parameters to test data.
- Centered rolling windows and backward fill may use future information.
- Create lag features only after sorting and within each entity/series.
- Use rolling-origin or expanding-window validation when appropriate.

**Leakage often produces excellent test scores that fail in real deployment.**

# Time-series preprocessing checklist

- Is the timestamp parseable, timezone-aware where needed, and correctly sorted?
- Is the frequency known—or is the series genuinely event-based and irregular?
- Have duplicate timestamps been interpreted rather than blindly dropped?
- Are missing values distinguished from missing time periods?
- Do resampling and smoothing choices match the measurement and task?
- Have outliers been checked against domain context?
- Are train/test splits chronological and all features available at prediction time?
- Can the raw data and each transformation be reproduced?

TEACHING NOTE: This checklist maps directly to Experiment 5.1: resampling and smoothing.

# Text data: what are we processing?



| | |
|---|---|
| **Granularity** | Character → word/subword token → sentence → document → collection. |
| **Sources** | Reviews, emails, social posts, chat logs, web pages, reports, transcripts. |
| **Tasks** | Sentiment analysis, spam detection, search, topic classification, entity extraction, summarization. |
| **Unstructured** | Meaning is carried by wording, context, order, punctuation, and sometimes formatting. |
| **Language variation** | Spelling, slang, morphology, abbreviations, multilingual text, emojis, and domain jargon. |

# Why raw text is messy



| | |
|---|---|
| **Formatting** | HTML tags, URLs, markup, extra whitespace, escaped characters. |
| **Encoding** | Broken characters, inconsistent Unicode, smart quotes, accented letters. |
| **Noise** | Typos, repeated letters, boilerplate, duplicated messages, bot-generated text. |
| **Language** | Contractions, inflections, compounds, abbreviations, mixed languages. |
| **Meaning** | Negation, sarcasm, ambiguity, named entities, and word sense depend on context. |
| **Labels & privacy** | Noisy annotations, personal data, sensitive content, and consent requirements. |

# Text cleaning: a task-aware pipeline



| | |
|---|---|
| **1. Preserve raw text** | Keep an untouched source column for audit and debugging. |
| **2. Normalize carefully** | Unicode normalization, whitespace cleanup, selected case normalization. |
| **3. Remove markup/noise** | Strip HTML or boilerplate only when it is not meaningful to the task. |
| **4. Tokenize** | Split into words, subwords, sentences, or characters depending on model/task. |
| **5. Optional filtering** | Stop-word removal, stemming, or lemmatization when appropriate. |
| **6. Represent & validate** | Vectorize or embed; inspect examples and test downstream performance. |

# Basic normalization in Python



```
                        import re
import unicodedata
from bs4 import BeautifulSoup

def clean_text(text):
    text = "" if text is None else str(text)
    text = unicodedata.normalize("NFKC", text)
    text = BeautifulSoup(text, "html.parser").get_text(" ")
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()

sample = "<p>Great   service!</p> https://example.com"
print(clean_text(sample))
```

## WHAT TO NOTICE

- Unicode normalization can unify visually equivalent forms; NFKC may also fold compatibility characters.
- HTML parsing is safer than deleting tags with a simplistic regex.
- URL removal is task-dependent: links may matter for spam or safety detection.
- This example preserves punctuation and letter case; add transformations only for a reason.

**Install beautifulsoup4 if using BeautifulSoup; inspect output on real samples.**

# Lowercasing, punctuation, numbers & symbols



| | |
|---|---|
| **Lowercase** | Reduces vocabulary size for many classification tasks, but can erase names/acronyms. |
| **Punctuation** | May be noise for bag-of-words; vital for sentence boundaries, emotion, or code. |
| **Numbers** | May be irrelevant in some reviews; essential for prices, dates, medical values, and IDs. |
| **Emojis** | Can express sentiment or intent; deleting them may remove signal. |
| **URLs / mentions** | May be removed, replaced with placeholders, or retained as features. |
| **Rule** | Do not apply a universal “remove everything non-alphabetic” rule. |

# Tokenization: turning text into units



| | |
|---|---|
| **Definition** | Split text into tokens—often words, subwords, sentences, or characters. |
| **Word tokenization** | Simple and interpretable; punctuation and contractions require rules. |
| **Subword tokenization** | Breaks rare/unseen words into pieces; common in modern transformer models. |
| **Sentence tokenization** | Separates sentences for summarization, chunking, or sentence-level analysis. |
| **Language matters** | Whitespace splitting fails for some scripts and languages; use language-aware tools. |
| **Validation** | Check contractions, hyphenated words, apostrophes, emojis, and multilingual examples. |

# Tokenization: simple vs language-aware



```
text = "The model isn't perfect, but it's useful!"

# Simple whitespace split (quick demo only)
print(text.split())

# NLTK word tokenizer (install nltk; download data as needed)
from nltk.tokenize import word_tokenize
tokens = word_tokenize(text)
print(tokens)
```

## WHAT TO NOTICE

- Whitespace split leaves punctuation attached and does not handle all contractions.
- A language-aware tokenizer applies more informed boundary rules.
- Tokenizer behavior depends on library/version and language resources.
- For transformer models, use the model's own tokenizer to match its vocabulary.

**Do not teach split() as a production-grade tokenizer.**

# Stop-word removal: what and why?



| | |
|---|---|
| **Definition** | Remove frequent function words such as “the”, “is”, and “of” from a token list. |
| **Potential benefit** | Can reduce vocabulary and noise in some traditional information-retrieval or bag-of-words tasks. |
| **Not universal** | Modern language models often need the original words and context. |
| **Danger: negation** | Removing “not”, “no”, or “never” can reverse sentiment or intent. |
| **Domain terms** | A frequent word may be important in law, medicine, or customer support. |
| **Best practice** | Use a task- and language-specific list; inspect the list before applying it. |

# Stop-word removal: preserve meaning



```
    from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

tokens = ["this", "movie", "is", "not", "good"]
filtered = [t for t in tokens if t not in ENGLISH_STOP_WORDS]
print(filtered)  # inspect whether "not" survives in your version/list

# Safer: define a reviewed list for your task
custom_stopwords = {"the", "a", "an", "is", "are"}
filtered = [t for t in tokens if t not in custom_stopwords]
```

## WHAT TO NOTICE

- Built-in stop-word lists vary; inspect exact contents in your environment.
- A generic list may remove task-critical terms or fail on local language usage.
- The custom list is intentionally small to illustrate control and review.
- Keep a copy of the original tokens so experiments can be compared.

**Always test negation examples such as “not safe”, “never worked”, and “no refund”.**

# Stemming vs lemmatization



## STEMMING

- Rule-based chopping of word endings; may produce non-words.
- Fast and often useful in search or classic text classification.
- Example: studies, studying → studi (algorithm-dependent).
- Can over-stem or under-stem; language-specific rules matter.
- Use when reduced vocabulary matters more than linguistic readability.

## LEMMATIZATION

- Maps a word to a dictionary base form (lemma), often using vocabulary and morphology.
- Example: studies → study; better lemmatizers use context/POS.
- Usually more linguistically meaningful, but may be slower or need extra resources.
- Can fail with slang, domain terms, names, or wrong part-of-speech assumptions.
- Use when readable normalized forms or morphology matter.

# Stemming and lemmatization in practice



```
   from nltk.stem import PorterStemmer, WordNetLemmatizer

words = ["studies", "studying", "cars", "better"]
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

for w in words:
    print(w, "-> stem:", stemmer.stem(w))

# WordNet lemmatizer often needs POS to do more
print(lemmatizer.lemmatize("studies", pos="v"))
print(lemmatizer.lemmatize("better", pos="a"))
```

## WHAT TO NOTICE

- Stemming is an algorithmic stem, not necessarily a valid word.
- Lemmatization depends on lexical resources; download NLTK WordNet data if needed.
- Part of speech (noun/verb/adjective) can change the lemma.
- “Better” may be lemmatized differently depending on POS and tool.
- Compare outputs on your corpus rather than assuming one method is always superior.

**For production, specify language and POS strategy and test domain vocabulary.**

# Text preprocessing pitfalls and fixes



| | |
|---|---|
| **Over-cleaning** | Removing punctuation, digits, or emojis can destroy signal → define rules per task. |
| **Negation loss** | Stop-word removal deletes “not” → preserve negators or skip stop-word filtering. |
| **Vocabulary mismatch** | Stemming conflates words or creates odd stems → compare with lemmatization/no normalization. |
| **Language mismatch** | English stop-word list applied to other languages → detect language and use suitable resources. |
| **Data leakage** | Vocabulary/IDF learned on all data → fit vectorizer on training data only. |
| **Duplicate/near-duplicate texts** | Same review in train and test inflates scores → deduplicate before splitting carefully. |
| **Privacy** | Names, phone numbers, and identifiers in logs → minimize, mask, and control access. |

# From cleaned text to model-ready features



| | |
|---|---|
| **Bag of words** | Counts tokens; simple baseline but ignores much word order. |
| **TF–IDF** | Downweights terms common across documents; useful for search and classic classifiers. |
| **N-grams** | Represent short sequences (e.g., “not good”) and preserve some local context. |
| **Embeddings** | Dense vector representations; often capture semantic similarity better than counts. |
| **Transformer input** | Use the model's tokenizer and expected normalization; avoid incompatible custom cleaning. |
| **Fit on training data** | Vectorizers learn vocabulary/statistics; do not fit them on validation/test text. |

# End-to-end mini example: text cleaning



```
                        texts = [
    "The service was not good!!!",
    "<b>Fast delivery</b> & great support",
    "Loved it 😊"
]

cleaned = [clean_text(t) for t in texts]
print(cleaned)

# Then choose tokenizer / optional filters for task
# X_train -> fit vectorizer
# X_valid, X_test -> transform only
```

## WHAT TO NOTICE

- Inspect original and cleaned strings side by side.
- The negation “not” and emoji may carry important sentiment information.
- Do not remove HTML entities or punctuation without checking the resulting text.
- Cleaning is not the same as feature extraction: vectorization comes afterward.

# How to decide whether a transformation is appropriate



## ASK BEFORE TRANSFORMING

- What is the prediction or analysis goal?
- Does this field carry meaning for that goal?
- Can the transformation use future/test information?
- What assumptions does the method make?
- Can I inspect examples where it fails?

## EVIDENCE TO KEEP

- Raw data and cleaned data side by side.
- Counts of missing values, duplicates, and altered records.
- Before/after plots for time series.
- Before/after text examples and token lists.
- Validation metrics and error analysis by subgroup or time period.

# Mini case A: electricity demand

CASE STUDY

| | |
|---|---|
| **Raw data** | Meter readings every 15 minutes; outages create gaps; duplicate messages occur. |
| **Task** | Forecast next-day hourly demand. |
| **Plan** | Parse timestamps and time zone; sort; resolve duplicate meter/time records; resample to hourly sum/mean according to units. |
| **Missingness** | Inspect outage lengths; interpolate only short plausible gaps and mark imputed points. |
| **Features** | Hour/day-of-week, lags, and trailing rolling statistics. |
| **Evaluation** | Train on earlier dates, test on later dates; verify no future information enters features. |

TEACHING NOTE: Ask pairs to justify the aggregation: is the source a cumulative meter, interval energy, or instantaneous power?

# Mini case B: customer review sentiment

CASE STUDY

| | |
|---|---|
| **Raw data** | Reviews include HTML, emoji, misspellings, and “not worth the price”. |
| **Task** | Classify positive vs negative sentiment. |
| **Plan** | Preserve raw text; normalize Unicode and whitespace; remove HTML; tokenize with a suitable tokenizer. |
| **Decision** | Keep negation, sentiment-bearing punctuation/emoji if useful; test whether stemming or lemmatization helps. |
| **Representation** | Start with TF–IDF word and bigram features; compare with an embedding-based model. |
| **Evaluation** | Split by customer/time if duplicates or repeated users can leak across splits; inspect false positives/negatives. |

# Quality assurance: test the pipeline



| | |
|---|---|
| **Time-series assertions** | Timestamps parse; rows sorted; expected frequency documented; no accidental duplicate keys. |
| **Text assertions** | No nulls where strings are expected; language/tool match; output samples reviewed. |
| **Regression tests** | Keep edge cases: daylight-saving boundary, long gap, negation, emoji, apostrophe, multilingual text. |
| **Reproducibility** | Pin package versions, record parameters, use deterministic rules where possible. |
| **Monitoring** | In production, track missingness, frequency changes, vocabulary drift, and performance drift. |

# Key takeaways



| | |
|---|---|
| **Time series** | Time order, frequency, missing intervals, resampling, smoothing, and leakage are central. |
| **Text** | Cleaning is task-dependent; tokenization is foundational, while stop-word removal/stemming/lemmatization are optional tools. |
| **Do not over-clean** | A transformation can remove signal just as easily as it removes noise. |
| **Validate** | Inspect raw and transformed data; test edge cases; compare downstream performance. |
| **Be reproducible** | Keep raw data, document decisions, and apply training-fitted transformations correctly. |

# Exit ticket: discuss or answer



- 1. Why is sorting by timestamp necessary before rolling-window calculations?
- 2. What is the difference between a missing timestamp and a missing value?
- 3. When might resampling from 1-minute to 1-hour data be useful—and what can it hide?
- 4. Why can centered rolling averages leak future information?
- 5. Why is removing “not” dangerous in sentiment analysis?
- 6. Give one difference between stemming and lemmatization.
- 7. Name one transformation you would avoid for a particular task, and explain why.

TEACHING NOTE: Use answers to identify concepts to revisit in the next practical session.